

작업자 생산성 평가 시스템을 함수와 메뉴 중심 구조로 확장하고 싶은데 내가 보낸 양식들 참고해서 4차 과제 조건에 맞게 설계해줘

좋아 형.

4차 과제는 기존 **작업자 생산성 평가 시스템**을 v3.0으로 확장해서, 단순 입력·조회 프로그램이 아니라 **데이터 저장까지 가능한 생산성 관리 시스템**으로 설계하면 돼.

구조는 이렇게 잡는 게 가장 좋아.

작업자 생산성 평가 시스템 v3.0

1. 작업자 정보 입력
2. 전체 작업자 조회
3. 생산성 분석
4. 파일 저장
5. 파일 읽기
0. 프로그램 종료

4차 과제 조건에 맞추려면 핵심은 **이중 리스트, 함수, 메뉴, 파일 저장, 예외 처리**야.

작업자 정보는 기초치러 이리 저어라 저어 시가 새사서 드글 트변 퍼가르 가가 따르 리스트에 저장하느

예를 들면 이렇게 설계할 수 있어.

<> Python



▶ 실행됨

```
workers = [
    ["김철수", 60, 3, 20.0, "S", "우수 작업자"],
    ["이영희", 45, 4.5, 10.0, "B", "-"]
]
```

이렇게 하면 한 명의 작업자 정보가 하나의 작은 리스트로 묶이고, 전체 작업자 목록은 큰 리스트 안에 저장돼. 4차 과제에서 요구하는 **2D List 구조**를 자연스럽게 만족할 수 있어.

기능은 함수로 나누면 돼.

<> Python



▶ 실행됨

```
add_worker()
show_workers()
analyze_workers()
save_file()
load_file()
```

`add_worker()`에서는 작업자 이름, 작업량, 작업 시간을 입력받고 생산성을 계산한 뒤 등급을 판정해서 `workers` 리스트에 추가해.

`show_workers()`에서는 이중 리스트를 반복문으로 순회하면서 전체 작업자 정보를 출력해.

`analyze_workers()`에서는 평균 생산성, 최고 생산성 작업자, 우수 작업자 수 같은 분석 결과를 출력할 수 있어.

+ 무엇이든 물어보세요

즉시 ▾



ChatGPT는 실수를 할 수 있습니다. 중요한 정보는 재차 확인하세요.

+ 무엇이든 물어보세요

즉시 ▾



전체적으로는 이렇게 확장하면 돼.

- 2차: 여러 작업자 데이터를 리스트에 저장
- 3차: 메뉴와 함수로 기능 분리
- 4차: 이중 리스트로 데이터 구조 개선 + 파일 저장 + 예외 처리 추가

이 구조로 설계하면 형이 보낸 양식의 4차 과제 조건인 **v3.0 기획, 메뉴/함수/전역변수 활용, 이중 리스트, 파일 저장, try-except 예외 처리**가 전부 자연스럽게 들어가.

📄 🍊 🔄 📁 ...

여러 작업자의 정보를 저장하기 위해 이중 리스트를 활용하고, 종료 시 txt 파일로 저장하는 구조를 설계할건데 방향 잡아줘

좋아 형.

4차에서는 **여러 작업자 정보를 따로따로 리스트에 저장하는 방식**보다, 한 명의 작업자 정보를 하나의 리스트로 묶고 전체를 큰 리스트에 저장하는 **이중 리스트 구조**로 가면 좋아.

방향은 이렇게 잡으면 돼.

Python

+ 무엇이든 물어보세요



▶ 실행됨

즉시 ▾



`save_file()`에서는 `open()` 또는 `with open()`을 사용해서 작업자 데이터를 `workers.txt` 파일로 저장해. `load_file()`에서는 저장된 파일을 읽어서 출력하되, 파일이 없을 경우 `try-except`로 오류를 처리하면 돼.

예외 처리는 최소 2곳 이상 넣는 게 좋아.

첫 번째는 작업량이나 작업 시간에 문자를 입력했을 때 발생하는 `ValueError` 처리야.

Python

```
try:
    amount = float(input("작업량: "))
    time = float(input("작업 시간: "))
except ValueError:
    print("숫자를 입력해야 합니다.")
```



▶ 실행됨

두 번째는 저장된 파일을 읽을 때 파일이 없으면 발생하는 `FileNotFoundError` 처리야.

Python

```
try:
    with open("workers.txt", "r", encoding="utf-8") as file:
        print(file.read())
except FileNotFoundError:
    print("저장된 파일이 없습니다.")
```



▶ 실행됨

전체적으로는 이렇게 확장하면 돼.

+ 무엇이든 물어보세요

즉시 ▾



각 위치 의미는 이렇게 정하면 돼.

```
[0] 작업자 이름  
[1] 작업량  
[2] 작업 시간  
[3] 생산성  
[4] 등급  
[5] 특별 평가
```

함수 구조는 이렇게 잡는 게 깔끔해.

```
add_worker() → 작업자 입력 후 workers에 append  
show_workers() → 이중 리스트 순회 출력  
analyze_workers() → 평균/최고 생산성 분석  
save_file() → workers 내용을 workers.txt로 저장
```

txt 저장은 프로그램 종료 메뉴를 선택했을 때 자동으로 실행되게 하면 좋아.

```
<> Python  
  
elif menu == "0":  
    save_file()  
    print("파일 저장 후 프로그램을 종료합니다.")  
    break
```

파일 저장 방식은 이런 구조야.

방향은 이렇게 잡으면 돼.

```
<> Python  
  
workers = []
```

여기서 `workers` 는 전체 작업자 목록이고, 작업자 한 명은 이런 식으로 저장해.

```
<> Python  
  
["김철수", 60, 3, 20.0, "S등급", "성과금 지급 대상"]
```

그래서 여러 명이 입력되면 전체 구조는 이렇게 돼.

```
<> Python  
  
workers = [  
    ["김철수", 60, 3, 20.0, "S등급", "성과금 지급 대상"],  
    ["이영희"]()45, 4.5, 10.0, "B등급", "일반 작업자"]  
]
```

각 위치 의미는 이렇게 정하면 돼.

```
[0] 작업자 이름  
[1]()
```

+ 무엇이든 물어보세요

즉시 ▾



Python



실행됨

파일 저장 방식은 이런 구조야.

Python



실행됨

```
with open("workers.txt", "w", encoding="utf-8") as file:
    for worker in workers:
        file.write(f"{worker[0]},{worker[1]},{worker[2]},{worker[3]},{worker[4]},{worker[5]}\n")
```

이렇게 하면 종료할 때 작업자 정보가 한 줄씩 저장돼.

김철수,60,3,20.0,5등급,성과급 지급 대상
이영희,45,4.5,10.0,8등급,일반 작업자



전체 방향은 이렇게 정리하면 돼.

입력 → 생산성 계산 → 등급 판정 → 이중 리스트 저장 → 조회/분석 → 종료 시 txt 저장



이 구조면 4차 조건인 **이중 리스트**, **append**, **중첩 for** 또는 **리스트 인덱싱 출력**, **파일 저장**, **함수 분리**가 자연스럽게 들어가.

📄 📌 🔍 🗑️ ↺ ...

+ 무엇이든 물어보세요

즉시 ▾



ChatGPT는 실수를 할 수 있습니다. 중요한 정보는 재차 확인하세요.